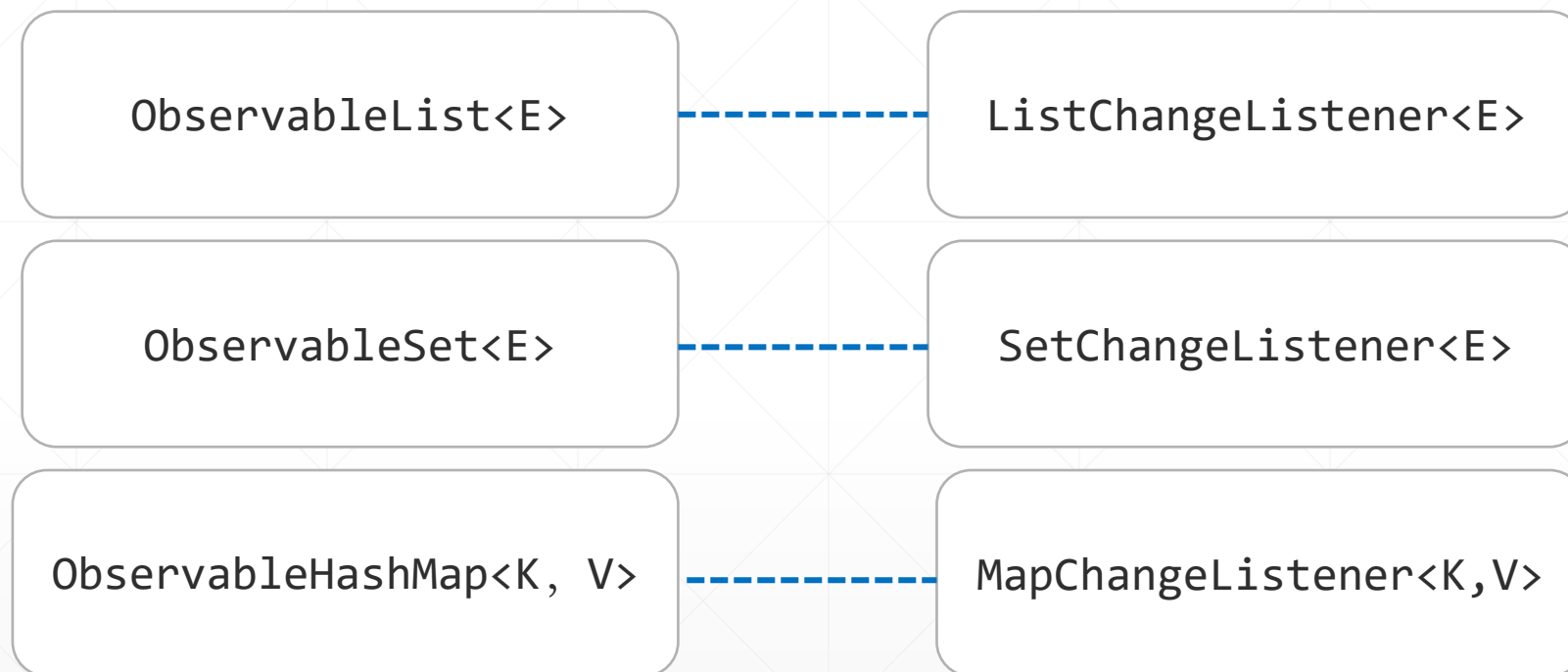




可观察的集合类型

北京理工大学计算机学院
金旭亮

JavaFX中三种可绑定集合



均为可观察的集合类型

在其接口中方法参数中可以
获取数据变动的相关信息

创建可观察的集合



使用FXCollections工具类中的相关方法，可以创建可观察的集合对象，有很多种集合类型，各有各的特点：

- ▶ `FXCollections.observableArrayList()`
- ▶ `FXCollections.observableIntegerArray()`
- ▶ `FXCollections.observableSet()`
- ▶ `FXCollections.observableHashMap()`
- ▶ `FXCollections.unmodifiableObservableMap()`
- ▶ `FXCollections.synchronizedObservableMap()`

ObservableList元素基本操作

可观察的集合（以ObservableArrayList为例）提供了诸多的方法操控集合：

```
// 创建一个包容两个字符串的可观察集合对象
ObservableList<String> list =
    FXCollections.observableArrayList( ...items: "one", "two");
System.out.println("List集合创建完毕: " + list);
// 添加两个字符串
list.addAll( ...elements: "three", "four");
System.out.println("添加完两个元素后: " + list);
// 移除[1,3)范围内的元素
list.remove(1, 3);
System.out.println("移除了[1,3) 范围内的元素后: " + list);
// 保留唯一的一个元素
list.retainAll( ...elements: "one");
System.out.println("只留下\"one\"一个元素, 其余全部移除: " + list);
```

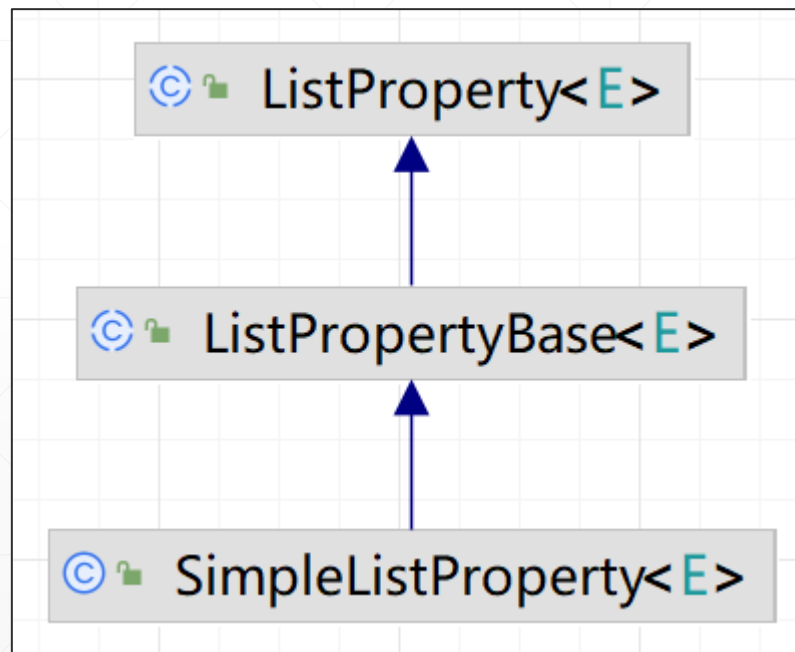
```
// 创建另外一个可观察的集合对象
ObservableList<String> list2 =
    FXCollections.observableArrayList( ...items: "1", "2", "3");
// 清空原有内容, 加入list2中的内容
list.setAll(list2);
System.out.println("清空List1, 将List2元素全部加入: " + list);
// 创建第3个集合
ObservableList<String> list3 =
    FXCollections.observableArrayList( ...items: "ten", "twenty", "thirty");

System.out.println("list2 : " + list2);
System.out.println("list3 : " + list3);
// 合并list2和list3两个集合
ObservableList<String> list4 = FXCollections.concat(list2, list3);
System.out.println("合并list2和list3之后: " + list4);
```

UseObservableList.java

可直接绑定的集合属性类型

JavaFX提供了一个比较独特的`ListProperty<E>`类型，它内部包装了一个`ObservableList<E>`，可以在两个集合之间直接建立绑定关系。



`ListProperty`是一个抽象类，真正使用的是`SimpleListProperty`类型。

构造`SimpleListProperty`实例时，需要向它的构造方法注入一个`ObservableList`集合。

可直接绑定的集合属性类型示例

```
public static void main(String[] args) {  
    System.out.println("实例化两个集合属性对象: lp1和lp2");  
    ListProperty<String> lp1 =  
        new SimpleListProperty<>(FXCollections.observableArrayList());  
    ListProperty<String> lp2 =  
        new SimpleListProperty<>(FXCollections.observableArrayList());  
    System.out.println("在两个ListProperty (lp1和lp2) 间建立单向绑定关系");  
    lp1.bindContent(lp2);  
    print(msg: "初始状态:", lp1, lp2);  
    lp2.addAll(...elements: "1", "2");  
    print(msg: "lp2加入两个元素后:", lp1, lp2);  
    System.out.println("取消单向绑定后, 再建立双向绑定关系");  
    lp1.unbindContent(lp2);  
    lp1.bindContentBidirectional(lp2);  
    //在左端集合添加元素  
    lp1.addAll(...elements: "3", "4");  
    print(msg: "lp1添加完元素后:", lp1, lp2);  
    //在右端集合添加元素  
    lp2.addAll(...elements: "5", "6");  
    print(msg: "lp2添加完元素后:", lp1, lp2);  
}
```

集合之间建立绑定关系之后，一个集合的变化，会影响到另一个集合.....

```
Run "C:\Program Files\Java\jdk-17.0.3.1\bin\java.exe" ...  
实例化两个集合属性对象: lp1和lp2  
在两个ListProperty (lp1和lp2) 间建立单向绑定关系  
初始状态: lp1: [], lp2: []  
lp2加入两个元素后: lp1: [1, 2], lp2: [1, 2]  
取消单向绑定后, 再建立双向绑定关系  
lp1添加完元素后: lp1: [1, 2, 3, 4], lp2: [1, 2, 3, 4]  
lp2添加完元素后: lp1: [1, 2, 3, 4, 5, 6], lp2: [1, 2, 3, 4, 5, 6]  
  
Process finished with exit code 0
```


理解ListProperty的对象引用



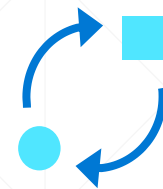
```
public class UnderstandListPropertyReference {
    public static void main(String[] args) {
        ListProperty<String> lp1 =
            new SimpleListProperty<>(
                FXCollections.observableArrayList());
        ListProperty<String> lp2 =
            new SimpleListProperty<>(
                FXCollections.observableArrayList());
        //看看lp1和lp2引用的是不是同一个集合对象
        System.out.println(lp1.get() == lp2.get()); //false
        lp1.bind(lp2);
        //看看lp1和lp2引用的是不是同一个集合对象
        System.out.println(lp1.get() == lp2.get()); //true
        lp1.addAll( ...elements: "One", "Two");
        System.out.println(lp2.get()); //[One, Two]
        lp2.set(FXCollections.observableArrayList( ...items: "1", "2"));
        System.out.println(lp1.get());//[1, 2]
        //由于绑定的限制，以下这句抛出运行时异常: A bound value cannot be set.
        // lp1.set(FXCollections.observableArrayList("3", "4"));
        lp1.unbind();
        //解绑后，lp1和lp2仍然引用同一个集合对象
        System.out.println(lp1.get() == lp2.get()); //true
        //但现在，lp1可以重新被设置了
        lp1.set(FXCollections.observableArrayList( ...items: "3", "4"));
    }
}
```

建立了绑定关系的两个
ListProperty，实际上是引用同
一个可观察的集合对象。

如果这个绑定关系是单向的，则仅
能在一端设置引用新的集合对象。

解绑后或建立的是双向绑定，则两
端都可以设置引用新的集合对象。

ListProperty与其它类型属性的组合应用



```
public class ListPropertyBindToStringProperty {  
    public static void main(String[] args) {  
        ListProperty<String> lp =  
            new SimpleListProperty<>(FXCollections.observableArrayList());  
        StringProperty initStr = new SimpleStringProperty( initialValue: "大小: ");  
        StringProperty desc = new SimpleStringProperty();  
        desc.bind(initStr.concat(lp.sizeProperty())  
            .concat( other: ", 为空? ")  
            .concat(lp.emptyProperty())  
            .concat( other: ", 内容: ")  
            .concat(lp.asString()));  
        System.out.println("在添加元素前: " + desc.get());  
        lp.addAll( ...elements: "张三", "李四");  
        System.out.println("在添加元素后: " + desc.get());  
    }  
}
```

级联多个方法调用，
完成字符串的拼接

```
Run  
"C:\Program Files\Java\jdk-17.0.3.1\bin\java.exe" ...  
在添加元素前: 大小: 0, 为空? true, 内容: []  
在添加元素后: 大小: 2, 为空? false, 内容: [张三, 李四]  
  
Process finished with exit code 0
```


跟踪集合变化

对于可以观察的集合，当集合中的元素有增删或变化时，会触发一个“集合已经修改”事件，通过给这个事件添加监听器，可以跟踪集合的变化。

```
Run
"C:\Program Files\Java\jdk-17.0.3.1\bin\java.exe" ...
集合元素有变化: [one, two, three]
集合元素有变化: [one, two, three, four, five]
集合元素有变化: [three, four, five]
集合元素有变化: [new value, four, five]

Process finished with exit code 0
```



```
public class UseInvalidationListener {
    public static void main(String[] args) {
        // 创建一个集合对象
        ObservableList<String> list =
            FXCollections.observableArrayList( ...items: "one", "two");
        //给集合添加一个监听器
        list.addListener(new InvalidationListener() {
            @Override
            public void invalidated(Observable observable) {
                System.out.println("集合元素有变化: "+observable);
            }
        });
        list.add("three");
        list.addAll( ...elements: "four", "five");
        list.remove(0,2);
        list.set(0, "new value");
    }
}
```

监听Invalidated事件，只能知道发生了变化，以及变化后集合的现状，却不能知道变化的详细信息。如果需要这个信息，可以添加另外一种监听器.....

了解集合变化的详细信息

对于可以观察的集合，当集合中的元素有增删或变化时，将触发一个ListChange事件，此事件关联一个Change对象，封装了更改信息。

我们可以给集合添加一个ListChangeListener，从而跟踪集合的修改情况。

```
interface ListChangeListener<E> {  
    void onChanged(Change<E> change);  
}
```

```
class Change<E> {  
    ObservableList<E> list;  
    boolean next();  
    boolean wasPermutated();  
    boolean wasRemoved();  
    boolean wasUpdated();  
    void reset();  
    boolean wasAdded();  
    int getPermutation(int index);  
    boolean wasReplaced();  
    int removedSize();  
    int addedSize();  
    int[] permutation();  
    ObservableList<E> list;  
    int from;  
    List<E> removed;  
    int to;  
    List<E> addedSubList;  
}
```

跟踪集合元素数目变化示例



```
public class TestMapChangeListener {  
    public static void main(String[] args) {  
        //创建可观察的Map集合  
        ObservableMap<String, Integer> map =  
            FXCollections.observableHashMap();  
        //添加MapChangeListener, 跟踪集合中元素的变化  
        map.addListener(  
            (MapChangeListener<String, Integer>) change  
                -> System.out.println("发生了变化: " + change));  
        //添加两个元素  
        map.put("one", 1);  
        map.put("two", 2);  
        //修改现有的元素  
        map.put("one", map.get("one") * 100);  
    }  
}
```

Run

"C:\Program Files\Java\jdk-17.0.3.1\bin\java.exe"

发生了变化: 1 added at key one
发生了变化: 2 added at key two
发生了变化: 1 replaced by 100 at key one

Process finished with exit code 0